

SIMATS ENGINEERING

ASSESSMENT TOOL 2

Name : C.Manohar
Reg No : 192372246
COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4303

COURSE OUTCOME COVERED

CO -2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool : Design Task

Weightage: 20%

QUESTIONS:

Total Marks: 20

Question 1: Responsive Navigation Bar Design - Marks(4)

Suppose that an e-commerce website requires a responsive navigation bar that adapts across desktop, tablet, and mobile devices.

- (a) Identify key components required in a navigation bar for an e-commerce website.
 - (b) Design a responsive navigation bar using appropriate CSS techniques.
 - (c) Demonstrate how the navigation adapts to smaller screens (e.g., hamburger menu).
 - (d) Evaluate the usability and suggest improvements for better user experience.
-

Question 2: Product Hover Effect - Marks(4)

Suppose that an e-commerce website wants to enhance user interaction through visual effects on product items.

- (a) Identify suitable CSS properties for hover effects.
 - (b) Suggest appropriate **CSS animation or transition** for product interaction.
 - (c) Design a hover effect for a product card.
 - (d) Evaluate performance and user experience considerations.
-

Question 3: Layout Selection (Flexbox vs Grid) - Marks(4)

Suppose that a dashboard layout needs to be designed for a web application.

- (a) Identify layout requirements of a dashboard (rows, columns, alignment).
 - (b) Compare **Flexbox and Grid** based on layout needs.
 - (c) Choose the most suitable layout technique and justify your choice.
 - (d) Evaluate the scalability and responsiveness of the chosen approach.
-

Question 4: Mobile-First Blog Layout - Marks(4)

Suppose that a blog page must be designed following a mobile-first approach.

- (a) Identify key sections of a blog layout.
 - (b) Design a mobile-first layout using CSS.
 - (c) Demonstrate how the layout scales for larger screens using media queries.
 - (d) Evaluate accessibility and readability of the design.
-

Question 5: JavaScript Event Handling for Dynamic Menu - Marks(4)

Suppose that a website includes a dynamic menu that responds to user interactions.

- (a) Identify appropriate JavaScript events for menu interaction.
- (b) Design event handling logic for menu toggle functionality.
- (c) Implement interaction for user actions (click/hover).
- (d) Evaluate performance and suggest improvements.

Code of Conduct:

I C.Manohar / 192372246 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Manohar

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Concept	25	Demonstrates complete understanding of design requirements and concepts	Good understanding with minor gaps	Basic understanding with some errors	Poor understanding or incorrect concepts
Design / Implementation	30	Well-structured, efficient, and responsive design implementation	Correct implementation with minor issues	Basic implementation with inconsistencies	Incorrect or incomplete implementation
Use of Tools (CSS/JS)	20	Effective and advanced use of tools/techniques	Appropriate use of tools	Limited or inconsistent use	Incorrect or minimal use
Analysis & Evaluation	15	Insightful analysis with strong justification and optimization	Good analysis with justification	Basic analysis with limited depth	Poor or no analysis
Documentation / Clarity	10	Clear, well-structured, and properly presented solution	Mostly clear with minor issues	Basic clarity, missing details	Poor or unclear documentation

ANSWERS

Question 1: Responsive Navigation Bar Design (4 Marks)

An e-commerce website requires a responsive navigation bar that adapts across desktop, tablet, and mobile devices.

(a) Key Components Required in the Navigation Bar

- Logo / brand name - links back to the homepage.
- Primary navigation links - Home, Categories, Deals, New Arrivals, etc.
- Search bar - lets users quickly find products.
- Account/login icon - access to the user's profile and orders.
- Cart icon - shows the number of items added, links to checkout.
- Hamburger toggle - appears on smaller screens to reveal the links/search in a collapsible menu.

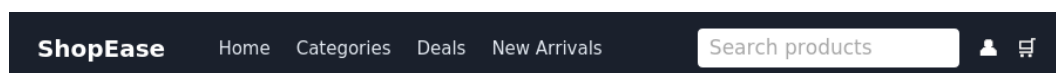
(b) Responsive Navigation Bar Design (CSS)

```
<nav class="navbar">
  <div class="logo">ShopEase</div>
  <ul class="nav-links">
    <li><a href="#">Home</a></li>
    <li><a href="#">Categories</a></li>
    <li><a href="#">Deals</a></li>
    <li><a href="#">New Arrivals</a></li>
  </ul>
  <div class="nav-right">
    <input class="search" placeholder="Search products">
    <span class="icon">account</span>
    <span class="icon">cart</span>
  </div>
  <button class="hamburger">&#9776;</button>
</nav>

.navbar { display: flex; align-items: center; justify-content: space-between;
          background: #1a202c; padding: 12px 24px; }
.nav-links { display: flex; list-style: none; }
.nav-links li { margin-right: 22px; }
.hamburger { display: none; }

@media (max-width: 640px) {
  .nav-links, .nav-right { display: none; }
  .hamburger { display: block; }
  .nav-links.active { display: block; width: 100%; }
}
```

Output - Desktop / tablet view



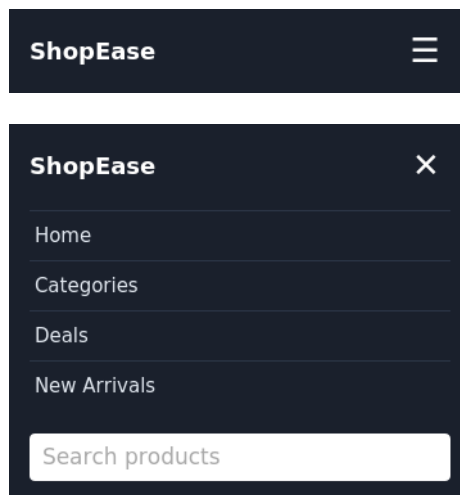
(c) Adapting to Smaller Screens (Hamburger Menu)

On screens narrower than 640px, the media query hides the link list, search bar, and icons, and shows only the hamburger button. JavaScript toggles an 'active' class on the link list when the button is clicked, sliding the menu open or closed:

```
const hamburger = document.querySelector('.hamburger');
const navLinks = document.querySelector('.nav-links');

hamburger.addEventListener('click', () => {
  navLinks.classList.toggle('active');
  hamburger.textContent = navLinks.classList.contains('active') ? '×' : '≡';
});
```

Output - Mobile view: menu closed (left) and after tapping the hamburger icon (right)



(d) Usability Evaluation and Suggested Improvements

The design keeps the navbar compact on mobile and gives full access to links on larger screens, which is good for usability. However, it can be improved by: adding a visible close (X) icon and slightly darkened background overlay when the mobile menu is open so users notice it clearly; ensuring the hamburger button has an aria-label and the menu list uses aria-expanded for screen-reader accessibility; keeping the search bar accessible even on mobile (e.g., as a separate search icon that expands into an input) rather than hiding it inside the menu; and adding a smooth slide/fade transition instead of an abrupt show/hide for a more polished feel.

Question 2: Product Hover Effect (4 Marks)

An e-commerce website wants to enhance user interaction through visual effects on product items.

(a) Suitable CSS Properties for Hover Effects

- transform (scale, translateY) - lifts or enlarges the card without affecting surrounding layout.
- box-shadow - adds depth, making the hovered card appear raised.
- opacity / filter - can dim a background image or brighten an overlay on hover.
- transition - animates the property changes smoothly instead of an instant jump.
- cursor: pointer - signals that the card is interactive/clickable.

(b) Suggested CSS Animation / Transition

transition: transform 0.3s ease, box-shadow 0.3s ease; is recommended. transform and box-shadow are preferred over animating width/height/top/left because they are handled efficiently by the browser's compositor (GPU-accelerated) and do not trigger expensive layout reflows.

(c) Hover Effect for a Product Card

```
.card {  
  background: #fff; border-radius: 8px; padding: 16px; text-align: center;  
  box-shadow: 0 1px 3px rgba(0,0,0,0.15);  
  transition: transform 0.3s ease, box-shadow 0.3s ease;  
}  
  
.card:hover {  
  transform: translateY(-8px) scale(1.04);  
  box-shadow: 0 12px 24px rgba(0,0,0,0.25);  
}
```

Output - Normal state (left) vs. Hover state (right)





(d) Performance and User Experience Considerations

Using transform and opacity keeps the effect smooth (60fps) since the browser does not need to recalculate page layout for every frame; animating properties like width, top, or margin should be avoided as they cause reflows and can feel janky on lower-end devices. The transition duration should stay short (200-350ms) so the interaction feels responsive rather than sluggish. On touch devices, :hover does not apply naturally, so an equivalent :active or tap state should be provided so mobile users get comparable feedback.

Question 3: Layout Selection - Flexbox vs Grid (4 Marks)

A dashboard layout needs to be designed for a web application.

(a) Layout Requirements of a Dashboard

- A fixed sidebar for navigation (Overview, Sales, Users, Reports, Settings).
- A row of KPI summary cards (Orders, Revenue, New Users) aligned evenly.
- A row (or grid) of larger widgets/charts below the KPIs.
- Both rows and columns need to align precisely - a genuinely two-dimensional layout.
- The layout must reflow into a single column on mobile without breaking alignment.

(b) Comparing Flexbox and Grid for This Layout

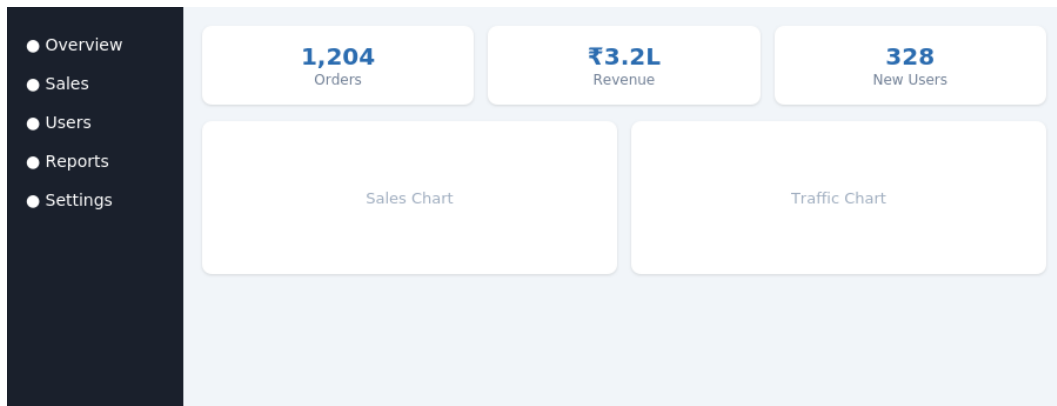
Aspect	Flexbox	CSS Grid
Dimension	One-dimensional (row OR column)	Two-dimensional (rows AND columns together)
Best for	Nav bars, button groups, card rows	Whole-page/dashboard skeletons
Alignment control	Good along a single axis	Precise control across both axes at once
Reordering widgets	Manual, via 'order' property	Easy, via named grid-template-areas
Nesting needed	Often needs nested flex containers	One grid can define the full structure

(c) Chosen Layout Technique and Justification

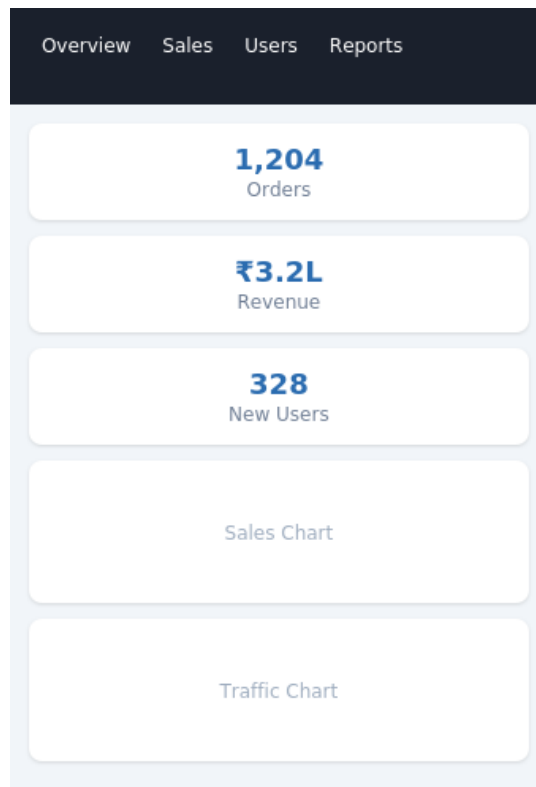
CSS Grid is the more suitable choice for the overall dashboard skeleton (sidebar + main content area + widget rows), because it is genuinely two-dimensional - the sidebar column and the KPI/widget rows can be defined together in one grid-template, keeping row heights and column widths aligned consistently. Flexbox is still used inside Grid for smaller one-dimensional groupings, such as arranging the KPI cards evenly in a row or aligning icons within the sidebar - this is the common real-world pattern of 'Grid for the page, Flexbox for the components'.

```
.dashboard { display: flex; }
.sidebar { width: 150px; }
.main { flex: 1; }
.kpis, .widgets { display: flex; }
.kpi, .widget { flex: 1; }
@media (max-width: 640px) {
  .dashboard, .kpis, .widgets { flex-direction: column; }
}
```

Output - Desktop view



Output - Mobile view: sidebar, KPI cards, and widgets stack vertically



(d) Scalability and Responsiveness Evaluation

The chosen approach scales well: adding a new KPI card or widget only means adding another flex/grid item - flex:1 on siblings keeps them evenly sized automatically, and grid's repeat()/minmax() functions (or auto-fit) would let the widget area accept new items without editing the CSS structure. A single max-width media query collapses the whole dashboard into a clean single-column layout for mobile, so the design remains usable from small phones up to wide monitors without separate mobile/desktop codebases.

Question 4: Mobile-First Blog Layout (4 Marks)

A blog page must be designed following a mobile-first approach.

(a) Key Sections of a Blog Layout

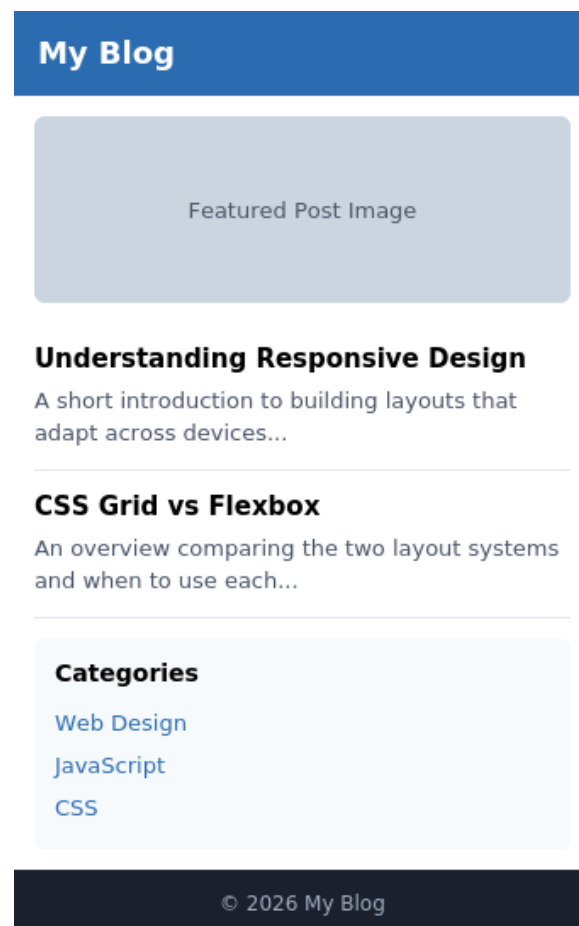
- Header - blog title/logo, and on larger screens a navigation menu.
- Featured/hero section - highlights the latest or most important post.
- Article list - post title, short excerpt, and 'read more' link for each post.
- Sidebar - categories, recent posts, tags, or an about/subscribe box.
- Footer - copyright, social links, secondary navigation.

(b) Mobile-First CSS

Mobile-first means writing the base (no media query) styles for the smallest screen first, then layering on enhancements for larger screens with min-width queries:

```
/* Base styles = mobile, single column, sidebar stacks below content */
header { padding: 14px; }
.hero { margin: 12px; height: 110px; }
.content { padding: 0 12px; }
.sidebar { margin: 12px; padding: 12px; }
```

Output - Mobile view (base styles, single column)

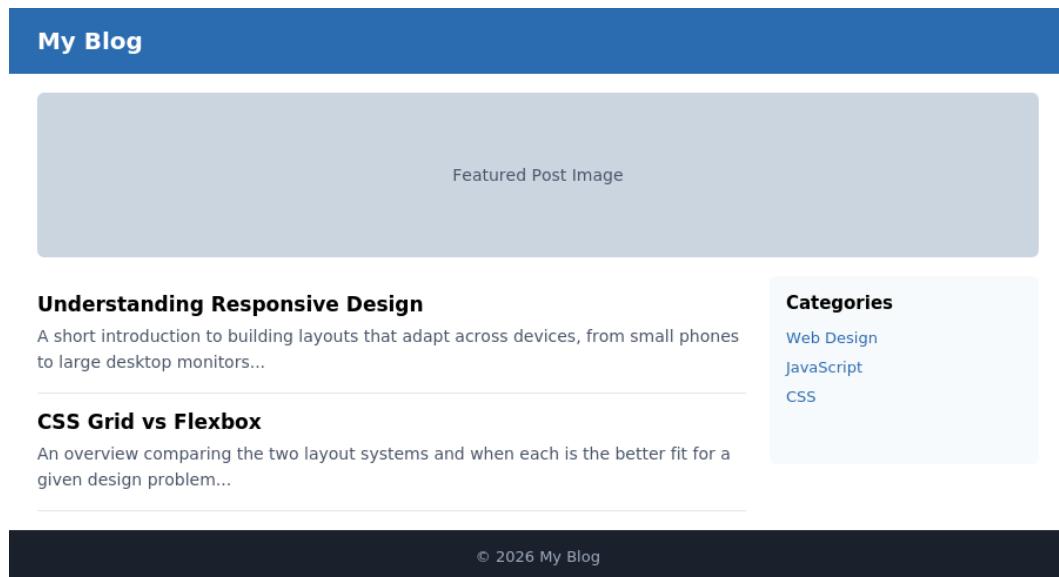


(c) Scaling Up for Larger Screens with Media Queries

```
@media (min-width: 768px) {
```

```
.layout { display: flex; padding: 0 24px; }  
.content { flex: 3; padding-right: 20px; }  
.sidebar { flex: 1; margin: 0; }  
.hero { height: 140px; margin: 16px 24px; }  
}
```

Output - Desktop view: content and sidebar sit side by side



(d) Accessibility and Readability Evaluation

The mobile-first base layout keeps text in a single readable column with a comfortable line length, which benefits small-screen readability the most. To strengthen accessibility further: use semantic tags (header, main, article, aside, footer) so screen readers can navigate sections directly; keep body text at least 16px with a line-height around 1.5-1.6; maintain sufficient colour contrast between text and background (especially the blue links on white); add descriptive alt text for the hero/featured image; and ensure the layout still works with the browser's text-size zoomed in, since fixed-height containers like the hero image can clip content if not handled with min-height instead of a hard height.

Question 5: JavaScript Event Handling for Dynamic Menu (4 Marks)

A website includes a dynamic menu that responds to user interactions.

(a) Appropriate JavaScript Events for Menu Interaction

- click - the primary event for toggling a dropdown/menu open or closed, works on both desktop and touch.
- mouseenter / mouseleave - used for hover-triggered menus on desktop (no bubbling, unlike mouseover/mouseout).
- keydown - for keyboard accessibility, e.g., closing the menu on Escape or navigating items with arrow keys.
- click on document (outside the menu) - used to close the menu when the user clicks elsewhere on the page.

(b) Event Handling Logic for Menu Toggle

On click, the handler toggles an 'open' class on the dropdown and updates an aria-expanded attribute; a second listener on the document closes the menu if a click happens outside it:

```
const menuBtn = document.querySelector('.menu-btn');
const dropdown = document.querySelector('.dropdown');

menuBtn.addEventListener('click', (e) => {
  e.stopPropagation();
  const isOpen = dropdown.classList.toggle('open');
  menuBtn.setAttribute('aria-expanded', isOpen);
});

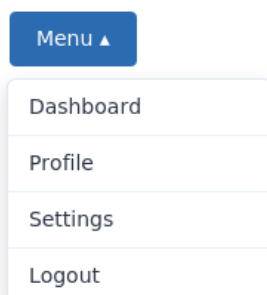
document.addEventListener('click', () => {
  dropdown.classList.remove('open');
  menuBtn.setAttribute('aria-expanded', 'false');
});
```

(c) Implementation for User Actions (Click)

Output - Menu closed (default state)



Output - After clicking the "Menu" button (open state)



(d) Performance Evaluation and Suggested Improvements

Attaching one listener to the button and one to the document is efficient for a single menu, but on a page with many menus, individual listeners on each item can add up; using event delegation (one listener on a common parent, checking `e.target`) scales better. Toggling a class (handled by CSS transitions) is preferred over changing inline styles in JavaScript, since it keeps animation performant and separates behaviour from presentation. Further improvements: debounce/guard rapid repeated clicks, close the menu on Escape key press for accessibility, and ensure focus moves into the menu when opened and returns to the button when closed, so keyboard users are not lost.